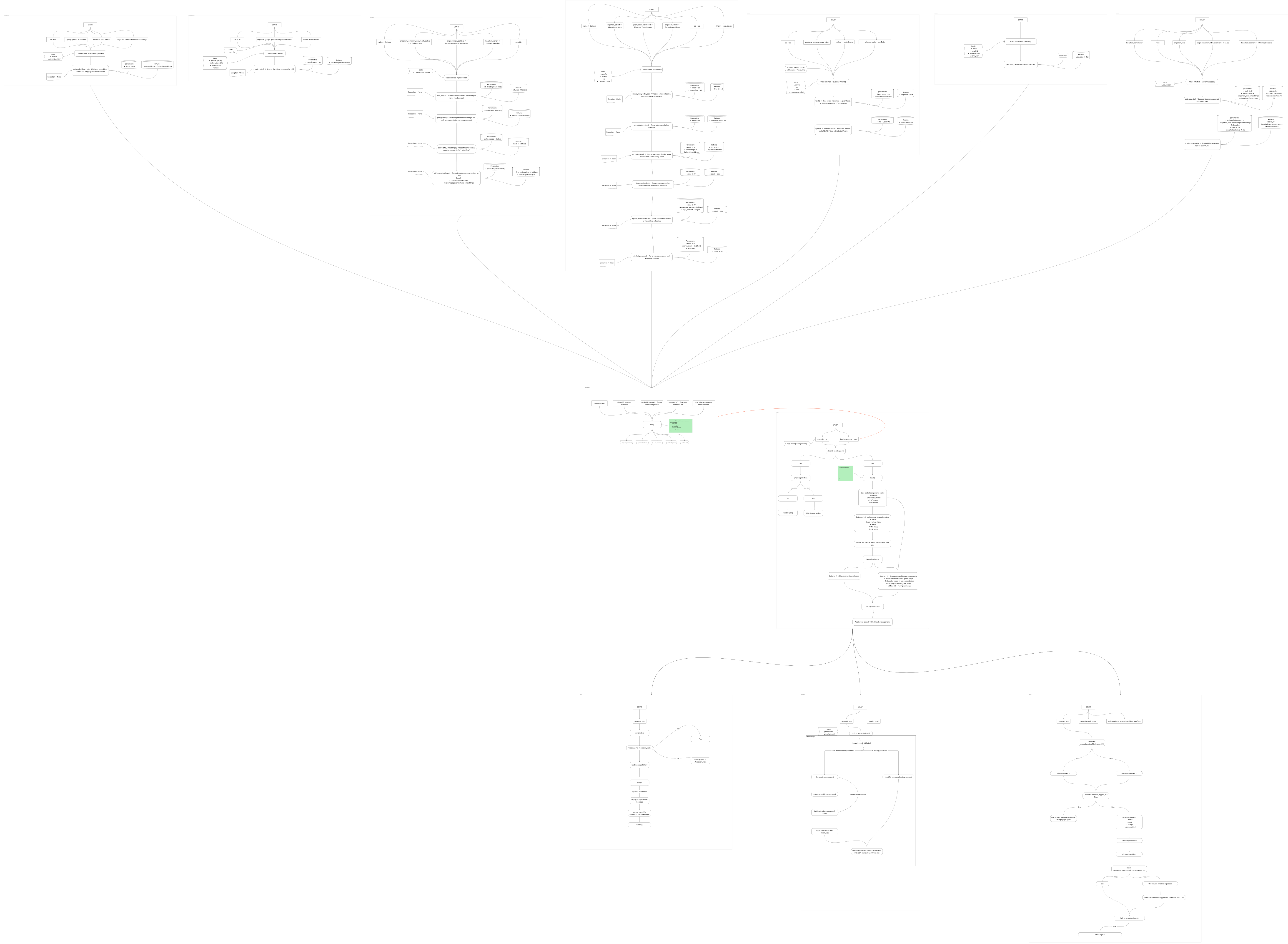
START
os → os
typing.Optional → Optional
dotenv → load_dotenv
langchain_cohere → CohereEmbeddings
loads
• .env file
• __cohere_apikey
Class initiated → embeddingModel()
get_emdedding_model → Returns embedding model from huggingface default model
Exception → None
parameters
• model_name
Returns
• embeddings → CohereEmbeddings
START
os → os
langchain_google_genai → GoogleGenerativeAI
dotenv → load_dotenv
loads
• .env file
loads
• google_api_key
• include_thoughts
• temperature
• verbose
Class initiated → LLM
get_model() → Returns the object of respective LLM
Exception → None
Parameters
• model_name → str
Returns
• llm → GoogleGenerativeAI
START
typing → Optional
langchain_community.document_loaders → PDFMinerLoader
langchain_text_splitters → RecursiveCharacterTextSplitter
langchain_cohere → CohereEmbeddings
tempfile
loads
• __embedding_model
Class initiated → processPDF
load_pdf() → Create a named temp file uploaded pdf -- stores in default path --
Parameters
• pdf → list[UploadedFile]
Returns
• pdf_load → list[str]
pdf_splitter() → Splits the pdf based on config's and split to documents to return page content
Parameters
• single_docs → list[str]
Returns
• page_content → list[str]
convert_to_emdeddings() → Feed the embedding model to convert list[str] → list[float]
Parameters
• splitted_docs → list[str]
Returns
• result → list[float]
pdf_to_emdeddings() → Compeletes the purpose of class by:
1. load
2. split
3. convert to embeddings
4. returns page content and embeddings
Parameters
• pdf → list[UploadedFile]
Returns
• final_embeddings → list[float]
• splitted_pdf → list[str]
Exception → None
START
typing → Optional
langchain_qdrant → QdrantVectorStore
qdrant_client.http.models → Distance, VectorParams
langchain_cohere → CohereEmbeddings
os → os
dotenv → load_dotenv
loads
• .env file
• apikey
• url
• __qdrant_client
Class initiated → qdrantDB
create_new_vector_db() → Creates a new collection and returns true on success
Exception → False
Parameters
• email → str
• dimension → int
Returns
• True → bool
get_collection_size() → Returns the size of given collection
Parameters
• email → str
Returns
• collection size → int
get_vectorstore() → Returns a vector collection based on collection name usually email
Parameters
• email → str
• embeddings → CohereEmbeddings
Returns
• db_store → QdrantVectorStore
delete_collection() → Deletes collection using collection name returns true if success
Parameters
• email → str
Returns
• result → bool
upload_to_collection() → Upload embedded vectors to the existing collection
Parameters
• email → str
• embedded_values → list[float]
• page_content → list[str]
Returns
• result → bool
similarity_search() → Performs vector results and returns list[results]
Parameters
• email → str
• query_vector → list[float]
• limit → int
Returns
• result → list
Exception → None
START
os → os
supabase → Client, create_client
dotenv → load_dotenv
utils.user_data → userData
schema_name = 'public'
table_name = 'user_data'
loads
• .env file
• url
• key
• __supabase_client
Class initiated → supabaseClient()
fetch() → Runs select statement on given table, by default statement ' * ' and returns
parameters
• table_name → str
• select_statement → str
Returns
• response → dict
upsert() → Performs INSERT if data not present and UPDATE if data exists but different
parameters
• data → userData
Returns
• response → dict
START
Class initiated → userData()
loads
• name
• email_id
• email_verified
• profile_icon
~~parameters~~
Returns
• user_data → dict
get_data() → Returns user data as dict
START
langchain_community
faiss
langchain_core
langchain_community.vectorstores → FAISS
langchain.docstore → InMemoryDocstore
loads
• is_db_present
Class initiated → vectorDataBase()
load_local_db() → Loads and returns vector db from givent path
parameters
• path → str
• embeddings → langchain_core.embeddings.embeddings.Embeddings
Returns
• vector_db → langchain_community.vectorstores.faiss.FAISS
initalize_empty_db() → Simply initializes empty new db and returns
parameters
• embeddingFunction → langchain_core.embeddings.embeddings.Embeddings
• index → str
• indesToDocStoreID → dict
Returns
• vector_db → langchain_community.vectorstores.faiss.FAISS
streamlit → st
qdrantDB → vector database
emdeddingModel → Cohere embedding model
processPDF → Engine to process PDF's
LLM → Large Language Models to chat
load()
creates the following instances and stores in st.session_state
• qdrant_client
• embedding_model
• pdf_processor
• processed_pdf_name
• large_language_model
• large_language_model
• processed_pdf_name
• pdf_processor
• embedding_model
• qdrant_client
START
streamlit → st
load_resources → load
page_config → page setting...
check if user logged in
No
Yes
Show login button
User clicks?
User clicks?
Yes
No
Run st.login()
Wait for user action
Executes load() function
load()
Gets loaded components status
• Database
• Embedding model
• PDF engine
• LLM models
Gets user info and stores in st.session_state
• Email
• Email verified status
• Name
• Profile image
• Login status
Deletes and creates vector database for each user
Setup 2 columns
Column - 1 → Display an welcome image
Column - 1 → Shows status of loaded components
• Vector database → red / green badge
• Embedding model → red / green badge
• PDF engine → red / green badge
• LLM model → red / green badge
Display dashboard
Application is ready with all loaded components
START
streamlit → st
vector_store
'messages' in st.session_state
Yes
Pass
No
Init empty list in st.session_state
load message history
prompt
If prompt is not None
display prompt as user message
append prompt to st.session_state.messages
working
START
streamlit → st
pandas → pd
• email
• placeholder_1
• placeholder_2
pdfs → Stores list [pdfs]
Inside loop
Loops through list [pdfs]
if pdf is not already processed
if already processed
Get result, page_content
toast file name as already processed
Upload embedding to vector db
Get len(embeddings)
Get length of vector per pdf name
append file_name and chunk_size
Update colledction size and dataframe with pdf's name along with its size
START
streamlit → st
streamlit_card → card
utils.supabase → supabaseClient, userData
Check for st.session_state["is_logged_in"]
True
False
Display logged in
Display not logged in
Check for st.user.is_logged_in if false
True
False
Pop an error message and throw to login page again
Declare and assign
• name
• email
• image
• email_verified
create a profile card
init supabaserClient
Check st.session_state.logged_into_supabase_db
True
False
pass
Upsert user data into supabase
Set st.session_state.logged_into_supabase_db = True
Wait for st.button(logout)
True
Make logout